CONFIDENTIAL

GAMEPADOS

THE COMPLETE BOOK

From Zero to Expert
Internal Study Plan & Reference Documentation

Prepared on: July 1, 2026

This is now the one document that teaches you everything about GamepadOS - from "what is a build" to the exact line of code that encrypts a controller packet. Read Part 0 first even if you think you know it; the rest of the book uses those words constantly and assumes you have them cold.

The project is two systems under one brand: the product (F:/hlooo/apps/ - the phone-to-PC gamepad itself) and the website (F:/hlooo/website/ - marketing, downloads, support desk). Parts 1–2 and 7–10 teach the product. Phases 0–6 (the original study plan) teach the website in deep, line-by-line detail.

Line numbers drift as you edit - every phase teaches you how to regenerate them with a grep. Content here reflects the state of the code as of 2026-07-01 (GRX encryption, two-mode gyro, and the v1.2.0 / v1.1.6 release).

PART 0 — THE DICTIONARY (READ THIS FIRST, NO EXCEPTIONS)

If you've never written software professionally, start here. Every word below gets used constantly for the rest of the book, and skipping this section is the #1 way people get lost in "advanced" docs. There's no shame in this - everyone starts here. This part has two halves: 0.1 is truly zero-assumed-knowledge (what a program even is), 0.2 is this-project's specific vocabulary (build, version, APK, etc.). Read both, in order, before Phase 0.

0.1 — How computers, programs, and the internet actually work

What a computer program is

A computer only ever does one thing: follow instructions, one at a time, extremely fast. Code (also called source code) is a list of those instructions, written in a programming language - a strict, simplified form of written language a computer can follow exactly (no ambiguity allowed, unlike English). A program (or application, or app) is a complete set of these instructions that does something useful - a game, a calculator, GamepadOS. - Programming language - there isn't just one; different languages exist for different jobs. This project alone uses four: Kotlin (Android app logic), C++ (fast native networking code), Python (the PC server), and TypeScript/JavaScript (the on-screen controller UI, which is really a tiny website). You don't need to know all four to understand the system - this book explains what each piece does in plain English, then points you at the exact code if you want to go deeper. - Syntax - the exact grammar rules of a language (where semicolons go, how to write a loop, etc.). Every language has different syntax for the same underlying ideas below. - Variable - a named "box" that holds a piece of data (a number, some text, a true/false value) that the program can read and change while it runs. E.g. left_stick_x = 128 stores the number 128 under the name left_stick_x. - Function (also called a method) - a named, reusable chunk of instructions that does one job, which you can "call" (run) from elsewhere in the code, often handing it some input (arguments / parameters) and getting back an output (a return value). Almost everything in this book - onSensorChanged, sendGamepadTelemetry, handle_frame - is a function name. - Conditional (if/else) - code that only runs when some condition is true ("if the button is pressed, do X, otherwise do Y"). This is how programs make decisions. - Loop

- code that repeats, either a fixed number of times or "until" some condition changes (e.g. "keep listening for network packets forever until the app closes"). - Class / object - a way of bundling related data and functions together under one name, modeling a "thing" (e.g. a GamepadPayload, a Ticket). You'll see this word constantly but don't need to master it to follow this book - think of it as "a named bundle of related stuff." - Data type - what kind of value a variable holds: a whole number (integer), a number with a decimal point (float), text (string), true/false (boolean), or "nothing" (null). Mismatched types are one of the most common bug sources - e.g. treating text "5" as if it were the number 5. - Array / list - an ordered collection of values under one name (e.g. a list of all currently-held buttons). - Bug - any place the code does something other than what was intended. Debugging is the process of finding and fixing one. - Comment - a note written in the code, for humans only, that the computer ignores completely (starts with // or # depending on the language). Good code uses comments to explain why something non-obvious was done, not what the code does (the code itself already says what).

Files, folders, paths, and "the codebase"

- File - one saved chunk of data on disk, with a name and an extension (the letters after the dot - .kt, .py, .tsx, .exe) that hints what kind of file it is and what program should open it.
- Folder (or directory) - a container that holds files and other folders, used to organize things. Folders can nest inside folders arbitrarily deep.
- Path - the "address" of a file or folder - the chain of folder names (separated by / or) you'd click through to reach it, e.g. F:/hlooo/apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt. An absolute path starts from the very top (a drive letter, here F:); a relative path is written starting from wherever you currently are.
- Code / source code - the actual text files a programmer writes (.kt, .py, .tsx, .cpp - the extension tells you the language). It's human-readable instructions. Nothing runs directly off this on the user's device (with two exceptions: Python and JavaScript, explained below) - most of it has to be compiled first.
- Codebase - all the source files for a project, together, usually inside one folder (here: F:/hlooo).
- Repository ("repo") - a codebase tracked by git (see below), so every change is recorded with history.

0.2 — Build & versioning vocabulary (this project's specifics)

Compiling, building, and running

- Compile - translating source code into a form a computer/chip can actually execute (machine code, or an intermediate form like Java bytecode). C++ and Kotlin are compiled languages. gamepad-engine.cpp is compiled by a C++ compiler (the NDK, see below) into a .so (a native library file for Android).
- Build - the overall process of turning source code into a finished, runnable artifact - compiling, bundling assets, signing, packaging. "Run the build" = "run the automated pipeline that produces the installable file." A build can succeed (BUILD SUCCESSFUL) or fail (a compile error, a missing file, etc.).
- Build artifact - the output of a build: an .apk (Android app), an .exe (Windows program), a dist/ folder of HTML/CSS/JS (a website). This is the thing you actually ship to a user.
- Debug build vs. Release build - two build modes with the same source code but different output:
- Debug - unoptimized, includes debugging info, much bigger file size, installs instantly for testing. Never ship this to real users.
- Release - optimized and shrunk (Android's shrinker is called R8/ProGuard - it deletes unused code and renames things to save space), signed with your real signing key, small file size. This is what you distribute.
- Concretely in this project: the debug APK is ~8.7 MB, the release APK is ~3.1 MB. Same features, very different packaging.
- Compile error - the compiler refuses to produce output because the code has a mistake (wrong syntax, a typo in a function name, a type mismatch). You fix the code, not the build tool.
- Runtime error / crash - the code compiled fine but does something illegal while running (e.g. divides by zero, calls a method on something that's null). These are why "it built successfully" ≠ "it works."

Interpreted vs. compiled — why Python and JavaScript are different

- Interpreted language - the source code is read and executed line-by-line by another program (an interpreter), with no separate compile step. Python (server.py) and JavaScript (the React UI, App.tsx after it's converted - see below) are interpreted-ish. You still "build" a Python .exe (via PyInstaller - it bundles the Python interpreter and your script together into one file), and you still "build" the React UI (via a tool called Vite that converts modern JS/TypeScript into a browser-runnable bundle).

- TypeScript - JavaScript with type-checking bolted on (catches bugs before you even run the code). App.tsx is TypeScript. The tsc command typechecks it (checks for type errors without producing a runnable file); vite build compiles/bundles it into real JavaScript the browser can run.

Versions and version numbers

- Version - a label that identifies which build of the software you have, so you (and the software's own update-checker) can tell old from new.
- Semantic-ish version string (1.2.0) - human-readable, shown to users. Usually MAJOR.MINOR.PATCH.
- versionCode (Android-specific) - a plain integer (12) that must strictly increase every release. Android - and this app's own in-app updater - compares these integers, not the human string, to decide "is this newer?" Forget to bump it and updates silently stop working.
- APP_VERSION (this project's PC server) - the equivalent string constant inside server.py, compared by the in-app "check for update" logic.
- Why THREE places must match (a real rule from this project's RELEASE.md): the PC installer script (.iss) has its own copies of the version (AppVersion, VersionInfoVersion) separate from server.py's APP_VERSION. If they drift, the admin panel that auto-detects versions shows a ?, or the updater offers a broken update. This is a real bug class, not theory - it bit this exact project.
- Signing key - release builds are cryptographically signed so the OS can verify "this update really came from the same developer as the last one." Android refuses to install an update signed with a different key than the one already on the device ("App not installed" error). This is why RELEASE.md warns: always sign with the same release.keystore.

Packaging formats you'll see in this project

- .apk - Android Package. A zip file containing compiled code, resources, and a manifest. What you install on an Android phone.
- .exe - a Windows executable. Either the program itself (GamepadServer.exe) or an installer that sets up the program (GamepadServer-Setup.exe).
- Installer vs. raw executable - the raw .exe just runs the program. An installer (built here with a tool called Inno Setup, from a .iss script) additionally: copies the program to Program Files, installs any drivers it needs, adds firewall rules, creates shortcuts, and registers an uninstaller. This project's installer wraps the raw server exe plus silently

installs a virtual-controller driver (ViGEmBus) and opens the firewall - so the user doesn't have to do any of that by hand.

Native code and cross-platform pieces

- Native code - code compiled directly to machine instructions for a specific CPU/OS combo (as opposed to Python/JS, which run through an interpreter). C++ is native. Native code is fast but not portable - you need a separate compiled copy per ABI (Application Binary Interface - essentially "CPU architecture"): arm64-v8a (modern phones), armeabi-v7a (older/budget phones), x86_64 (emulators, some tablets/Chromebooks).
- NDK (Native Development Kit) - Android's toolchain for compiling C++ into .so libraries the Android app can call into.
- JNI (Java Native Interface) - the bridge that lets Kotlin/Java code call into compiled C++ code and vice versa. Every function that crosses that bridge needs a matching name and signature on both sides - a very common source of "compiles fine, crashes at runtime" bugs if the names don't line up exactly.
- WebView - a browser engine embedded inside a native app. This project's Android app is a thin native shell that hosts the React controller UI inside a WebView - so the buttons/sticks you see are actually a tiny website running inside the app, talking to the native Kotlin/C++ layer through a JavaScript bridge.

0.3 — Internet, servers & data vocabulary (needed for both halves of the project)

How the internet actually works (needed for the website phases)

- Operating system (OS) - the base software that manages a computer's hardware and runs everything else on top (Windows, Android, macOS, Linux). Every program in this project ultimately runs on top of an OS.
- Internet - a huge network connecting computers worldwide so they can send each other data. A website and a web server talk to each other over it using rules called protocols.
- IP address - a numeric address identifying a specific computer on a network (e.g. 192.168.1.42). Port - a number (0–65535) identifying which program on that computer to talk to - one computer can run many programs that each listen on a different port (this project's PC server listens on port 7777).

- HTTP - the standard protocol web browsers and web servers use to talk to each other: the browser sends a request ("give me this page" / "here's this form data"), the server sends back a response. HTTPS is the same thing, encrypted in transit.
- Request / Response, GET / POST - a GET request asks for data (loading a page); a POST request sends data (submitting a form). The method (GET/POST/PUT/DELETE) says what kind of action is being requested.
- Status code - a 3-digit number in every HTTP response summarizing what happened: 200 = success, 404 = not found, 401 = not logged in, 500 = server crashed while handling it. You'll see these constantly in Phase 2.
- API (Application Programming Interface) - a defined set of "questions you're allowed to ask" a program, and the answers it gives back - usually over HTTP, exchanging data as JSON (a simple, universal text format for structured data, e.g. {"name": "Akhil", "count": 5}). This project's backend exposes an API; the website and the phone app are both clients of it.
- Frontend vs. backend - the frontend is what runs in the user's browser/device (what they see and click). The backend is the server-side program that stores data and enforces rules, which the frontend talks to over the API. This project's website has a clear split: website/frontend/ (static pages) and website/backend/ (the Express/Node.js API server).
- Database - a program specialized for storing and querying structured data reliably (this project uses PostgreSQL in production and SQLite for local testing - both are "relational databases," meaning data lives in tables with rows and columns, like a very powerful spreadsheet with rules).
- ORM (Object-Relational Mapper) - a library that lets you read/write database rows using normal programming-language objects instead of writing raw database query language by hand (this project uses Prisma). Phase 1 covers this in depth.
- Browser - the program that renders websites (Chrome, Safari, the Android WebView - a browser engine embedded inside another app, used here to host the controller UI inside the native Android app).
- Cookie - a small piece of data a website asks the browser to store and automatically resend on every request to that site - this is how "staying logged in" works without you re-entering your password on every click.

Networking, servers, and encryption — the basics

- Client / Server - the client asks for something, the server responds. Here: the phone is the client sending controller input; the PC runs the server that receives it and drives the virtual controller.

- UDP - a network protocol used for the phone→PC input stream. Chosen because it's low-latency (no "did you get that?" handshake overhead per packet, unlike TCP) - ideal for gaming input where an occasional dropped packet matters less than delay.
- Packet - one discrete chunk of data sent over the network. This project's input packet is a fixed 20 bytes: timestamp, buttons, triggers, stick positions, and an auth token.
- Latency - the delay between doing something (tilting the phone) and its effect appearing (the game reacting). Lower is better; this project targets a few milliseconds over USB.
- Encryption - scrambling data so only someone with the right key can read it. This project added GRX (an X25519 + AES-GCM encrypted handshake) so the input stream can't be read or spoofed by someone else on the same Wi-Fi.
- Handshake - the initial exchange of messages two devices use to agree on encryption keys before real data flows.

Sensors (for the gyro-steering feature)

- Sensor fusion - combining multiple raw sensors (gyroscope + accelerometer) into one reliable reading. A gyroscope alone drifts over time; fusing it with gravity (from the accelerometer) keeps it accurate.
- Gimbal lock - a mathematical blind spot where a particular way of describing 3D rotation (Euler angles: roll/pitch/yaw) breaks down at certain orientations - this project hit exactly this bug (steering "locked up" when the phone was held vertical) and fixed it by switching to a different, lock-free formula.
- Filter / smoothing - reducing sensor noise/jitter without adding too much delay. This project uses a "1€ filter," a well-known adaptive smoothing algorithm.

0.4 — Tools & workflow vocabulary

Tools and workflows you'll see referenced

- git - version-control software: records every change to the code as a "commit," lets you go back, compare, and collaborate without overwriting each other's work.
- Deploy - publishing a build somewhere users/servers can reach it (a website going live, a new APK becoming available to download).
- CI/CD - automated pipelines that build/test/deploy code without a human doing it by hand each time (this project uses one for the website's "keep the free server awake" job).

- Admin portal - an internal (password-protected) web page for managing the running system - here, it manages support tickets and which app/server version is the "active" one users get offered.

What you're actually building (the mental model)

You own two completely separate systems that share only a brand and a download link:

1. The product (F:/hlooo/apps/) - the real GamepadOS: phone → UDP packets → PC server → virtual controller, tuned for a few ms of latency. A hard-realtime input system on its own toolchain (Kotlin + C++ on the phone, Python on the PC). Covered in Phases 7–10.
2. The website + support platform (F:/hlooo/website/) - an ordinary CRUD web app that markets the product, distributes its installers, and runs a support desk. Covered in Phases 0–6 (the original, most deeply detailed part of this book).

The subsystems you'll master, in dependency order: 1. Data layer - Prisma schema is the vocabulary of everything. Learn it first. 2. API server - server.js, one file, ~38 routes, the spine of the backend. 3. Auth - sessions + roles guarding those routes; auth.js + middleware in server.js. 4. Admin portal - admin.html, a single-file vanilla-JS SPA consuming the API. 5. Webhook / inbound email + frontend site + infra/deploy - the edges connecting to the outside world.

How to use this plan

- Each phase has a goal, concepts (beginner→advanced), exact files/lines to read, hands-on exercises on your own code, external topics, and a self-check.
- Pro insight callouts are the non-obvious gotchas from your own codebase - the stuff that bites you at 2am. Read every one twice.
- Do the exercises. Reading alone ≈ 30% retention; changing your own code and watching it break/work ≈ 90%.
- Version-control first. If F:/hlooo/website isn't a git repo yet, run git init, commit a baseline, and work on a branch so you can always git diff / git checkout . to undo experiments.
- Keep a running NOTES.md answering every self-check question in your own words. Explaining your system back to yourself is what converts "I read it" into "I own it."

PHASE 0 — FOUNDATIONS & ORIENTATION

Goal: Boot the backend locally, open the admin portal, submit a ticket through the real form, and name every top-level file and what it does - without looking anything up.

Concepts (beginner → advanced)

Concept	Difficulty	Where
Static site vs dynamic backend split	beginner	README.md
Express app + app.listen boot	beginner	server.js:10, :1115, end of file
Environment-based config (12-factor)	beginner	server.js top; package.json; .env.example
Build/start scripts & deploy-time prisma db push	intermediate	backend/package.json scripts
Two-schema reality: Postgres (prod) vs SQLite (dev)	intermediate	prisma/schema.prisma vs prisma/schema.local.prisma
Multi-page static site (MPA) vs SPA	intermediate	frontend/vite.config.js inputs

Read & trace (in order)

1. README.md - the deploy runbook. Your map.
2. backend/package.json - prestart runs prisma db push, start runs node server.js. Schema sync happens via npm, not in code.
3. backend/.env.example and frontend/.env.example - every config the two halves read.
4. server.js top (express(), trust proxy, PORT) and bottom (health route :1115, app.listen).

Local setup — the exact SQLite path (do this once)

The prod schema targets Postgres; for local work use the SQLite mirror:

```
cd F:/hlooo/website/backend
cp .env.example .env                          # then edit DATABASE_URL="file:./prisma/
npx prisma db push --schema=prisma/schema.local.prisma   # creates/syncs dev.db
npm install
npm start
# prestart db push + listen
# confirm:  curl http://localhost:3000/    → health JSON
```

> ⚠️ schema.local.prisma uses provider = "sqlite" and lags prod - it may be missing csatRating, csatFeedback, and TicketMessage.attachments. Anything touching those works in prod but can break locally. Don't trust "works locally" for those columns (this becomes the key lesson in Phases 1 & 5).

Hands-on exercises

1. Boot it locally using the steps above. Watch prestart → db push → listen fire in order.
2. Submit a real ticket end to end. Run the frontend (cd frontend && npm run dev), open /contact.html, submit, and watch the ticket land in the backend logs / DB. See the cross-origin POST arrive.
3. Regenerate the route map (and keep it honest). Run:

   grep -nE "app.(get|post|put|delete)(" backend/server.js

Compare it line-by-line to Reference Table A at the bottom of this doc. Line numbers drift every time you edit server.js - re-running this grep is how you stop the plan from rotting. Bonus: this teaches you the whole route inventory faster than reading does. 4. Inventory drill. Without notes, write one sentence per file for: server.js, auth.js, inbound.js, prisma/schema.prisma, admin.html, login.html.

External topics

- Node + npm scripts: prestart/start lifecycle, --env-file-if-exists.
- Express basics: express(), app.listen(PORT, '0.0.0.0', cb) - and why bind to 0.0.0.0 (so Railway's proxy can reach the container).

- Vite: dev server (ESM + HMR) vs vite build → dist/.

🔑 Pro insight

> node server.js directly ≠ npm start. Only npm start runs the prestart hook that does prisma db push. Launch the server any other way and no schema sync happens - the root cause of the "column may be missing in prod" defensiveness you'll meet in Phase 5. Burn this in now.

Self-check

- What are the two glue-config strings, and what breaks if each is wrong?
- Why must the server bind to 0.0.0.0?
- Which npm script syncs the DB schema, and when does it run?
- Name the two disjoint systems under the GamepadOS brand and the only thing they share.

PHASE 1 — THE DATA LAYER (PRISMA + POSTGRES/SQLITE)

Goal: Read schema.prisma fluently, explain every relation and onDelete rule, predict what a prisma.ticket.update({ include }) returns, and add a column safely.

> Why first? The schema is the vocabulary of the whole backend. Every route is CRUD on these models. Master the nouns before the verbs.

Concepts (beginner → advanced)

Concept	Difficulty	Where
Relational modeling (the models)	beginner	schema.prisma
ORM - rows as objects	beginner	every prisma.. in server.js
Scalar types & nullability (String?)	beginner	schema.prisma
@default, @unique, @default(uuid()) PKs	intermediate	schema.prisma
One-to-many & named relations ("AssignedTickets")	intermediate→advanced	schema.prisma
include vs select (projection)	intermediate	reply/list handlers in server.js
upsert, count, filter DSL (startsWith/ mode/not)	intermediate→advanced	settings, dashboard, match handlers
Referential actions (Cascade / SetNull)	advanced	schema.prisma relation fields
Indexes (@@index)	advanced	schema.prisma
Prisma error codes P2002 / P2025	advanced	server.js team-create & assign handlers
db push vs migrations	advanced	package.json + comments in server.js

Read & trace

1. schema.prisma in full. For each model say aloud: its PK, its relations, its onDelete.
2. Three flows in server.js: ticket create (:506), admin reply with nested message write (:628), dashboard read with include + projected assignee (:554).

Hands-on exercises

1. Add a column end to end. Add Ticket.internalPriorityNote String?, npx prisma db push, then write it from an admin route and display it in admin.html. Feel the schema→DB→API→UI loop.
2. Map every relation by hand. Draw the ER diagram: every FK, every onDelete. Then answer: what happens to an admin's tickets, sessions, and audit rows when you delete that admin? (Three different answers - verify against the schema.)
3. Provoke the error codes. curl two admins with the same email → P2002 → 409. PUT a reply to a non-existent ticket id → P2025 → 404.
4. Connect CSAT to the schema. Note csatRating / csatFeedback on Ticket. These are written by GET /api/csat (:1067) and POST /api/csat/:ticketId/feedback (:1084) - and they're exactly the columns missing from schema.local.prisma. Predict: against a stale local DB, which two routes throw? (This is the concrete payoff of the Phase 0 warning.)

External topics

- Prisma: schema language, datasource/generator, findMany/findUnique/create/update/upsert/count, nested writes, include vs select, where-filter DSL, P2002/P2025. Read the Relations and Referential actions docs.
- Relational fundamentals: PKs, FKs, unique constraints, what an index does to a lookup, one-to-many cardinality.
- UUID v4: why high-entropy IDs let id: { startsWith: ref } matching work from a short email subject ref.
- db push vs migrations: db push has no version history and no rollback.

🔑 Pro insights

> Everything is String-typed on purpose - status, priority, role, tags (comma-separated lowercase), csatRating. No enums, no scalar arrays - for cross-DB portability (Postgres ↔ SQLite). Consequence: the database enforces no valid-value constraint; all validation lives in server.js.
>
> AuditLog.adminName is denormalized on purpose. Because AuditLog.admin is onDelete: SetNull, deleting an admin nulls adminId but the human-readable name survives - history outlives the account. Identity choices are feature-driven. AdminSession uses the random cookie token as its PK (auth = one PK lookup). Setting uses its semantic key as PK (a true key/value table). Neither is boilerplate.

Self-check

- Which models have indexes, and what query justifies each?
- What does deleting an Admin do to their sessions / assigned tickets / audit rows?
- Why is Ticket.reply kept even though the full thread lives in TicketMessage?
- How does select: {id, name} on assignee act as a security boundary?

PHASE 2 — THE API SERVER CORE (REQUEST LIFECYCLE, ROUTING, MIDDLEWARE)

Goal: Trace any request through server.js line by line, explain middleware ordering, know which status every outcome maps to, and add a route correctly.

Concepts (beginner → advanced)

Concept	Difficulty	Where
Middleware pipeline & ordering	beginner	cors→json near top; 404 last :1120
REST routing (:id params)	beginner	ticket routes
HTTP status codes as a contract	beginner	201/400/401/403/404/409/503 across handlers
HTML vs JSON responses	beginner	CSAT HTML :1063; sendFile :1100; health JSON :1115
Body parsing & limits	intermediate	express.json 2mb; route-local urlencoded : 1081
Zod validation	intermediate	ticketSchema + safeParse in :506
Route-level middleware composition	intermediate	requireAdmin/requireOwner; multer at :302
CORS dynamic allowlist	intermediate	cors options near top; * override on /api/ version :1056
Multipart uploads (multer diskStorage)	intermediate	:127-136, :302
Static serving with forced download	intermediate	:1092 Content-Disposition: attachment
In-memory state (presence, settings cache)	advanced	presence Map; settings cache

Read & trace — the canonical lifecycle

Trace the admin-reply flow end to end - the richest one. Open server.js:628-687 and walk every step: 1. cors → express.json → requireAdmin → handler. 2. Validate reply, load ticket (404 if missing). 3. Build update: set reply/repliedAt, clear hasNewReply, nested-create TicketMessage, conditionally set status/resolvedAt/firstResponseAt, auto-assign if unassigned, append CSAT links if resolving (:666). 4. prisma.ticket.update with include. 5. await sendMailSafe (this one is awaited - see Phase 5). 6. audit(...) fire-and-forget +

sseBroadcast('ticket:update', ...). 7. Respond with {success, ticket, emailed}; P2025 → 404 else 500.

Hands-on exercises

1. Write the curl, trace the path. A valid POST /api/support/ticket (:506) and one with message too short. For each, write every middleware/handler line touched and the exact status.
2. Add an authenticated read endpoint. GET /api/admin/stats behind requireAdmin returning counts via prisma.ticket.count. Practice middleware composition + the {success:true,...} contract.
3. Break middleware order on purpose. Temporarily move the 404 catch-all (:1120) above a real route; watch everything below it 404. Move it back. Express routing is first-match in registration order.
4. Trace the upload→download pair. curl a file to POST /api/admin/upload (:302), get back /downloads/, GET it, confirm the forced Content-Disposition (:1092).
5. See a CORS preflight. curl -X OPTIONS -H 'Origin: https://evil.test' -H 'Access-Control-Request-Method: POST' http://localhost:3000/api/support/ticket -i and inspect which Access-Control-* headers come back vs an allowed origin. This is the concrete root of the Phase 6 "works in curl, fails in browser" skill.

External topics

- Express middleware model: (req,res,next), ordering, why body parsing precedes routes reading req.body. Learn Express 5 specifics (differs from v4).
- HTTP: 200/201/400/401/403/404/409/500/503, Content-Type, Content-Disposition.
- CORS: preflight, Access-Control-Allow-Origin, why a dynamic origin fn returns cb(null, false) (not an Error).
- Zod: safeParse vs parse. multer 2.x: diskStorage, fileSize limits.

🔑 Pro insights

> CORS returns cb(null, false), not an Error, for disallowed origins. The request still runs - it just lacks the Access-Control-Allow-Origin header, and the browser blocks the response. Requests with NO Origin (curl, server-to-server, the inbound webhook) are allowed through - which is why the webhook relies on a URL token, not CORS. Trailing slashes are stripped from the allowlist but not from incoming Origin (browsers never send one). A FRONTEND_URL configured with a trailing slash would otherwise never match. Classic footgun. /api/version deliberately sets Access-Control-Allow-Origin: * because the Android WebView fetches it from an arbitrary origin. It's the one intentionally fully-open endpoint alongside static /downloads.
>
> ⚠️ SECURITY - admin attachments are served publicly and unauthenticated. POST /api/admin/upload writes into downloads/ (multer destination → downloadsDir, :130), and /downloads is an express.static mount with no auth (:1092). So any support attachment is downloadable by anyone who knows/guesses the filename - filenames are Date.now()-random, i.e. security-by-obscurity only. If you ever upload sensitive customer files, this is a real exposure. Consider an authenticated /api/admin/attachments/:id route instead. The uploads/ directory is dead. There are two dirs on disk - downloads/ and uploads/ - but multer writes to downloads/. uploads/ is empty/legacy; nothing references it. (Confirming a directory is unused, and safely removable, is the mastery here.)
>
> No global error-handling middleware and no rate limiter. Each async route try/catches itself; the only size guards are the 2mb JSON limit and multer's file limit. If you add a route, you own its error handling. The settings cache and presence map are in-process Maps, explicitly justified by "single-instance backend." Horizontally scale and presence goes wrong per-instance and the settings cache serves stale values. A hidden scaling constraint baked into the design.

Self-check

- Why does the 404 catch-all only work registered last?
- res.send vs res.json vs res.sendFile vs res.redirect - where is each used?
- A disallowed origin calls POST /api/support/ticket - does the handler run? Does the browser see the response?
- Where does an admin file upload physically land, and who can download it?

PHASE 3 — AUTHENTICATION & AUTHORIZATION (THE MODEL + ITS WEAKNESSES)

Goal: Explain the entire auth mechanism - scrypt hashing, opaque sessions, cookie flags, the two roles, the Bearer back door - AND articulate its real weaknesses well enough to decide whether to fix them.

Concepts (beginner → advanced)

Concept	Difficulty	Where
requireAdmin middleware (the choke point)	beginner	server.js auth block
Role-based authz (requireOwner)	beginner	auth block
Page guards are redirects, not security	beginner	/admin :1100, /admin/login : 1107
scrypt hashing + per-password salt	intermediate	auth.js
Opaque session tokens + server-side storage	intermediate	auth.js + session lookup in server.js
HttpOnly / SameSite / Secure cookie flags	intermediate	cookie-set in server.js
First-run setup gating	intermediate	auth-state :311, setup :328, login.html
Session expiry & active-check on every request	intermediate	session lookup + revoke on deactivate
Bearer / ADMIN_PASSWORD bypass	intermediate→advanced	requireAdmin Bearer branch
Trust proxy (makes Secure work)	advanced	server.js:10 area
timingSafeEqual	advanced	auth.js
CSRF exposure	advanced	absent; SameSite is the only defense

Read & trace

1. auth.js in full: hashPassword (scrypt + 16-byte salt → scrypt:salt:hash), verifyPassword (length-check then timingSafeEqual), newSessionToken (32 random bytes → 64 hex), parseCookies.
2. The auth block in server.js - cookie/session/guard. The heart of it.

3. login.html - the first-run setup UI: it calls /api/admin/auth-state and branches between setup (no admins yet) and login. Trace that branch; it's where the very first owner is born.
4. auth.test.js - the assertions document the exact guarantees (roundtrip, wrong/empty password, salt uniqueness, garbage never throws).

Hands-on exercises

1. Hash a password by hand. In a node REPL: require('./auth'), hash "test1234" twice - outputs differ (different salts), both verify true; a wrong password verifies false.
2. Watch the cookie. Log in via login.html, DevTools → Application → Cookies → gp_admin: confirm HttpOnly, SameSite=Lax, Max-Age, and whether Secure is set (it won't be on localhost http). Map each flag to the cookie-set code.
3. Exercise the Bearer back door. Set ADMIN_PASSWORD in .env, curl an admin route with Authorization: Bearer . Works as a synthetic owner. Then POST /api/admin/me/password with it and watch it refuse (because req.admin.id is null). Then hit GET /api/admin/stream with the same Bearer and confirm the back door also opens the live SSE event stream.
4. Force a logout. Deactivate an admin, then make a request with their old cookie - rejected because admin.active is re-checked on every request.

External topics

- scrypt (RFC 7914): why memory-hard KDFs beat fast hashes; what a salt defeats.
- crypto.timingSafeEqual: timing side-channels; why === leaks match info via response time.
- Opaque sessions vs JWTs: the revocability trade-off (DB read per request vs instant logout).
- Cookie security: HttpOnly (XSS), SameSite=Lax (CSRF), Secure (no plaintext leak), trust proxy behind a TLS-terminating PaaS.
- CSRF: what SameSite=Lax does and doesn't cover; what a CSRF token / Origin check would add.

🔑 Pro insights — strengths AND weaknesses

STRENGTH - textbook password storage. scrypt + 16-byte per-password salt + timingSafeEqual, dependency-free and tested. verifyPassword length-checks before timingSafeEqual (which throws on unequal-length buffers). Genuinely correct.

STRENGTH - opaque sessions beat JWTs here. 256-bit random lookup keys, no forgeable claims. Logout deletes the row; deactivation deletes the admin's rows; active is re-checked every request, so a disabled admin is locked out instantly. WEAKNESS - the Bearer/ADMIN_PASSWORD back door is your softest spot. Full owner power, bypasses scrypt+sessions, the comparison is a plain === (not timingSafeEqual), it reaches even the SSE stream, and Bearer actions log adminName:'API token' with adminId:null - no audit trail tying actions to a person. If ADMIN_PASSWORD leaks or is weak, the portal is silently compromised.

WEAKNESS - no brute-force protection. No rate limiting/lockout on /api/admin/login or /setup. scrypt slows offline cracking, not online guessing. (Login does return a single generic error - no user-enumeration - but /auth-state reveals whether any admins exist.)

WEAKNESS - no CSRF tokens. Sole defense for state-changing POSTs is SameSite=Lax + JSON expectation. Low risk (internal/same-origin) but deliberately thin. GOTCHA - Secure cookie is conditional, set only when req.secure or x-forwarded-proto === 'https'. Correct behind Railway; behind a misconfigured proxy the cookie could ride plaintext HTTP. This is why trust proxy is load-bearing. DESIGN - page guards are not the security boundary. GET /admin just redirects and serves a static SPA with no secrets. The real protection is every /api/admin/* route independently enforcing requireAdmin.

Self-check

- Walk the full login→request→logout flow naming every DB write and cookie op.
- Why is verifyPassword's length-check before timingSafeEqual necessary?
- Name three concrete weaknesses of the Bearer back door (including what it reaches).
- Why does deactivating an admin log them out immediately rather than at token expiry?
- What does SameSite=Lax protect against, and what does it not?

PHASE 4 — THE ADMIN PORTAL SPA (ADMIN.HTML)

Goal: Read the single-file vanilla-JS SPA, explain the api() wrapper, optimistic updates, SSE live push, presence, the Quill editor, and the broadcast feature - and add a new admin action wired to a backend route.

> One file, ~1300 lines: inline CSS design system, an HTML shell, and all JS. No framework, no build step. Mental model: mutate a global, then call renderList()/renderDetail().

Concepts (beginner → advanced)

Concept	Difficulty	Where (in admin.html)
Cookie auth - no tokens in JS, react to 401	beginner	api() + logout()
Module-global state + re-render model	beginner	top-of-script globals
DOM via innerHTML template rendering	beginner	esc(), renderList, renderDetail
Role-based UI (owner vs agent)	beginner	loadMe, delete/team gating
esc() XSS escaping (+ trusted-HTML exception)	intermediate	esc(); unescaped m.body
Centralized api() fetch wrapper + file:// mock	intermediate	api()
Filter/search/sort list pipeline	intermediate	filtered() + renderList
Quill rich-text + draft persistence	intermediate	reply editor mount
Optimistic UI with rollback	advanced	setPriority / assignTicket / setStatus
SSE live push + 20s poll fallback	advanced	connectStream + poll
Presence/collision heartbeat	advanced	startPresence + beforeunload
Background broadcast (queued mass email)	advanced	broadcast modal → POST /api/admin/broadcast

Read & trace

1. api() - the single choke point: 401 → redirect, parse JSON, throw on !res.ok || !data.success. Note the giant file:// mock that fabricates data when opened from disk; only the final fetch branch runs on a real origin.
2. Bootstrap: DOMContentLoaded → loadMe → Promise.all([loadTeam, loadCanned, loadSettings]) → stats pane → loadTickets → connectStream.

3. View-and-reply: selectTicket (note auto-ack open→in_progress + auto-markRead), renderDetail, Quill mount in setTimeout(0), startPresence, sendReply.

Hands-on exercises

1. Add an admin action. A "Snooze 1 day" button in renderDetail that optimistically sets snoozedUntil and PUTs to the backend - following the save-prev/apply/revert-on-error shape of setStatus. Internalize optimistic updates with rollback.
2. Trace an SSE event. With the portal open, submit a ticket from another browser and watch ticket:new fire: chime, toast, liveRefetch. Add a console.log to see the payload.
3. Break esc() to prove XSS. Temporarily render a user's name without esc(), submit a ticket with as the name, watch it fire, then revert. Confirm m.body is intentionally unescaped (trusted Quill HTML).
4. Drive the broadcast worker pool. Trigger a broadcast to a small contact set, observe the immediate {queued:true} response, then find the tally in the Activity log. Read the broadcast handler (server.js:1002-1034) and explain the concurrency-6 worker pool and the 500-recipient cap - a fire-and-forget background job whose results land in AuditLog.

External topics

- Fetch API with credentials (same-origin cookies); why no Authorization header is needed.
- EventSource / SSE on the client: named events, reconnect, one-directional.
- DOM innerHTML rendering vs virtual DOM: focus/scroll loss, re-binding, why guards exist.
- XSS: how string-concatenated HTML is exploitable; what esc() escapes.
- Quill 1.3.6: why it must mount after innerHTML commits (the setTimeout(0)).
- Web APIs: navigator.sendBeacon (presence on unload), Notifications, Web Audio (chime), matchMedia.

🔑 Pro insights

> Response contract is {success:true, ...} - api() throws unless both res.ok AND data.success. Mutating endpoints return the updated ticket; the client swaps TICKETS[i] = data.ticket to stay authoritative. api() throws Error('auth') on 401 so each catch can suppress its own toast for auth failures. Selecting a ticket has two hidden side effects: an 'open' ticket auto-moves to 'in_progress', and any new email reply is auto-marked read. hadNewReply is captured before markRead because markRead clears the flag. Reply/note bodies are inserted UNescaped - a deliberate trust boundary: admin-authored Quill HTML is trusted; user plaintext is escaped. attachments is a JSON string, JSON.parse'd at render time.
>
> Role gating here is UX only. Owners see Team/Activity/Broadcast/Delete; the server enforces the real boundary via requireOwner. The client merely hides controls. The empty detail pane is a live stats dashboard computed entirely client-side from TICKETS - there's no stats endpoint. Avg first-response derives from firstResponseAt vs createdAt.

Self-check

- What does api() do on a 401, and how does that propagate through every catch?
- Explain the optimistic-update shape and why it skips revert on auth errors.
- Why is m.body not escaped while name/email are?
- Why must Quill mount inside setTimeout(...,0)?
- SSE vs the 20s poll: which is primary, what's the fallback, when does the poll skip?
- What are the broadcast worker pool's concurrency and recipient limits, and where does its result get recorded?

PHASE 5 — THE INBOUND-EMAIL WEBHOOK + CSAT ROUND-TRIP (END TO END)

Goal: Explain the complete round-trip - how an admin reply leaves the portal, how a user's email reply finds the exact ticket, the three-tier matching, the reliability design - AND the CSAT satisfaction loop. The most intricate subsystem; budget extra time.

The end-to-end picture

```
ADMIN replies in portal
   │  server.js sends via Brevo (api.brevo.com/v3/smtp/email)
   │  stamps:  Reply-To: ticket+<ticket.id>@<EMAIL_REPLY_DOMAIN>
   │           In-Reply-To/References: user's last Message-ID
   │           if resolving: appends 👍/👎 CSAT links → /api/csat?ticketId=…&rating
   ▼
USER
```

receives email — subject has short [#a1b2c3d4] ref, Reply-To is per-ticket │ USER hits Reply → addressed to ticket+@reply. ▼ DNS MX for reply. → Brevo inbound (SMTP hop) │ Brevo parses MIME → JSON {items:[...]} → HTTP POST → /api/email/inbound?token=TOKEN ▼ server.js:738 validate ?token === INBOUND_WEBHOOK_TOKEN (401 mismatch / 503 unset)

```
RESPOND 200 IMMEDIATELY  ← beat Brevo's few-second timeout
   │  then, in a detached async IIFE, per item:
   ├─ parseInboundItem (inbound.js) → {fromEmail, subject, messageId, body, recipie
   ├─ loop guard: skip if fromEmail === SENDER_EMAIL.toLowerCase()
   ├─ MATCH ticket, 3 tiers:
   │     1. UUID in recipient (ticket+<uuid>@) → findUnique
   │     2. [#ref]
```

in subject → findFirst id startsWith ref │ 3. newest ticket where email == fromEmail (insensitive) ├─ no match → forward raw email to ADMIN_EMAIL_USER (don't lose it) ├─ PRIMARY write (long-standing columns only): hasNewReply=true, reopen if resolved, │

nested-create TicketMessage {sender:'user', via:'email', body} ├─ BEST-EFFORT 2nd write (own try/catch): lastEmailMessageId └─ sseBroadcast('ticket:userReply') + notify ADMIN_EMAIL_USER

```
CSAT loop (separate):
   USER clicks 👍/👎 → GET /api/csat?ticketId&rating (:1063) → writes csatRating →
   USER submits free-text → POST /api/csat/:ticketId/feedback (:1081) → writes csat
```

Concepts (beginner → advanced)

| Concept | Difficulty | Where | |---|---|---| | Webhook = reverse API / HTTP callback | beginner | server.js:738 | | Brevo + protocol chain (SMTP then HTTP) | beginner | inbound.js; INBOUND_EMAIL_SETUP.md | | Payload shape & body-precedence parsing | intermediate | inbound.js + inbound.test.js | | Webhook security via shared-secret token | intermediate | server.js:738-744 | | Email-to-ticket threading & 3-tier matching | advanced | inbound.js + match block in webhook | | Ack-fast / fire-and-forget background processing | advanced | server.js:746 area | | Idempotency / loop & duplicate guards | advanced | self-mail guard; MessageId stored but NOT deduped | | Third-party HTTP integration w/ timeout | advanced | sendMailSafe + AbortSignal.timeout | | CSAT capture round-trip | intermediate | :1063, :1081; link-gen at :605 & :666 |

Read & trace

1. inbound.js in full: parseInboundItem body precedence (ExtractedMarkdownMessage → RawTextBody → stripHtml(RawHtmlBody), 10k cap), findTicketUuid, findSubjectRef, normalizeMessageId.
2. inbound.test.js - documents the exact Brevo payload shape and expected behavior. Your spec.
3. server.js:738-835 - the webhook handler: ack-first, 3-tier match, split write.
4. INBOUND_EMAIL_SETUP.md - operator runbook (MX records, webhook registration, env vars, matching precedence).

Hands-on exercises

1. Simulate Brevo without Brevo. curl a realistic {items:[...]} payload (copy the shape from inbound.test.js) to /api/email/inbound?token= with recipient ticket+@reply.yourdomain. Watch a TicketMessage appear and hasNewReply=true. Tier-1 matching.

2. Test all three tiers. Repeat with (a) UUID recipient, (b) no UUID but [#ref] subject, (c) neither but From matches an existing ticket's email. Then one matching nothing → confirm it forwards to ADMIN_EMAIL_USER.
3. Prove ack-first. console.time/timeEnd around the response vs the DB write; confirm the 200 returns before persistence. Explain why (Brevo's timeout).
4. Provoke the duplicate gap. POST the same item twice → two TicketMessage rows (no MessageId dedup). Decide whether you'd fix it (unique index on MessageId).
5. Close the CSAT loop. Resolve a ticket, find the 👍/👎 links in the outbound email, click "good", confirm csatRating is written, submit the feedback form, confirm csatFeedback. Note: against a stale local SQLite DB these routes throw - the Phase 0/1 lesson made concrete.
6. Hit the email diagnostics. GET /api/admin/email-status (verifies the Brevo key via /v3/account) and POST /api/admin/email-test. These are the first endpoints you'll reach for when "email is broken in prod."

External topics

- Webhooks: client/server inversion; ack-fast/background-process reliability; why a short provider timeout forces "respond before persist."
- Brevo Inbound Parsing: the SMTP→parse→HTTP chain (docs linked in inbound.js).
- DNS MX records: how mail for a subdomain routes to Brevo; why the reply subdomain must differ from the sending domain (avoid loops).
- RFC 5233 plus-addressing (ticket+@): carrying state in the address.
- Email threading headers (RFC 5322): Message-ID, In-Reply-To, References.
- HMAC webhook signatures: what you'd gain over a URL token (integrity + non-leakage).

🔑 Pro insights

Two distinct hops are easy to conflate: SMTP (user→Brevo, governed by MX records) and HTTP (Brevo→your webhook, governed by the registered URL). The reply subdomain MUST differ from the sending domain or you loop.

Ack-first is intentional and has a cost. Brevo gives only a few seconds and logs webhookFailed on timeout, dropping the reply. So you respond 200 then persist. Cost: any exception in the background save is merely console.error'd and lost forever - Brevo already got 200, no retry, no dead-letter queue. The DB write is deliberately split in two. The comment documents a real past bug: a single atomic update that also wrote lastEmailMessageId threw entirely when that newer column was missing in prod (skipped db push), dropping the whole reply. Splitting + try/catch guarantees the message survives even when the optional column is absent. The direct payoff of the Phase 0 db push insight.

No idempotency key. MessageId is parsed and stored, but only for outbound threading (lastEmailMessageId), never checked to prevent duplicates. A Brevo re-delivery duplicates the message.

3-tier matching degrades gracefully because mail clients are unreliable (some strip Reply-To, some rewrite subjects). Tier 3 (newest ticket by sender email) can misattribute if the user has multiple open tickets - a known accuracy trade-off. normalizeMessageId exists because a malformed stored Message-ID placed raw into In-Reply-To/References can make Brevo reject the send - breaking every reply after the first. It only attaches a header it can prove well-formed.

The reply email is deliberately AWAITED while almost every other email is fire-and-forget - because a silently-dropped human reply is worse than ~300ms latency. The API returns emailed so the UI can warn. Know which sends block and why.

SENDER_EMAIL = EMAIL_FROM_ADDRESS || ADMIN_EMAIL_USER, and the self-mail loop guard compares fromEmail === SENDER_EMAIL.toLowerCase(). Set EMAIL_FROM_ADDRESS with mixed case and the .toLowerCase() is what keeps the guard working - a fragile derivation worth knowing. CSAT can fire twice. Both PUT /tickets/:id (status→resolved, :605) and reply-with-resolve (:666) independently build CSAT links, so a user could get them twice - a dedup gap analogous to the inbound idempotency gap. backfillLegacyReplies runs on EVERY boot, not just once (in the listen callback). It's idempotent - filtered by messages: { none: { sender:'admin' } } - which is the actual lesson: safe to run every startup, not a one-time migration.

Self-check

- Name the two hops (SMTP vs HTTP) and what governs each.
- Why respond 200 before writing to the DB, and what's the cost?
- Why is the DB write split into two updates? Which past bug does that prevent?
- Walk all three matching tiers and tier 3's failure mode.
- Why is the admin-reply email awaited while the confirmation email is fire-and-forget?
- Trace the full CSAT round-trip from the email link to csatFeedback.

PHASE 6 — FRONTEND SITE + INFRA, DEPLOY & KEEPALIVE

Goal: Explain the static MPA, build-time env inlining, the cross-origin contract, and the full deploy + keepalive lifecycle - and diagnose a "form works in curl but not the browser" bug in seconds.

Concepts (beginner → advanced)

Concept	Difficulty	Where
Static site, no server logic		
beginner	all frontend/*.html	
Vite (dev server vs vite build→dist)	beginner	vite.config.js; frontend/ package.json
Committed binaries served as downloads	beginner	.gitignore; static mount server.js:1092
CI/CD vs scheduled keep-alive job		
beginner	.github/workflows/ keepalive.yml	
Free-tier cold starts (sleep-on-idle)	intermediate	keepalive.yml + health route :1115
Env config - import.meta.env/VITE_* (build-time)	intermediate	frontend/js/config.js
Form POST to API via fetch	intermediate	
frontend/js/contact.js		
Reverse proxy / trust proxy	intermediate	server.js:10 area
HTTP email API instead of SMTP (platform constraint)	advanced	sendMailSafe

Read & trace

1. frontend/js/config.js - the API_BASE resolution (VITE_API_BASE → localhost:3000 → placeholder) and wireDownloadLinks().

2. frontend/js/contact.js - the only place the static site sends user data: validate (mirrors backend), POST JSON, swap thank-you state, mailto fallback on failure.
3. .github/workflows/keepalive.yml - cron */5, curls URLs, succeeds if either responds; the cold-start comment.
4. README.md deploy section - Railway (backend) + Vercel (frontend) + FRONTEND_URL for CORS.

Hands-on exercises

1. Reproduce the CORS deploy bug. Locally set FRONTEND_URL to the wrong origin, submit the contact form from the browser → CORS failure. Confirm the same request via curl succeeds (no Origin → allowed). Fix FRONTEND_URL → works. Own the most confusing deploy-day bug.
2. Prove build-time inlining. Build with VITE_API_BASE=https://example.test, grep dist/ JS for example.test, find it hard-coded. Explain why changing the backend URL needs a rebuild, not an env flip on the host.
3. Trace a download link. Click [data-dl="android"], confirm it points at /downloads/GamepadOS.apk, follow to express.static (:1092). Then check the EXE: /api/version and the static URLs reference GamepadServer-Setup.exe - confirm the committed filename in backend/downloads/ matches exactly, or the download 404s. (Both GamepadServer.exe and GamepadServer-Setup.exe exist on disk - the version manifest must point at the right one.)
4. Trigger keepalive manually via workflow_dispatch; confirm it curls the health route; read the "best-effort" comment.

External topics

- Vite: import.meta.env, VITE_ prefix, public/ verbatim copy, multi-page rollup inputs. VITE_ vars are inlined at build and are public - never put a secret there.
- CORS deep: why the browser blocks cross-origin reads; why curl/Postman bypass it.
- GitHub Actions: cron syntax, workflow_dispatch, default-branch-only schedules.
- PaaS hosting: Railway free-tier idle sleep / cold starts; why trust proxy is mandatory behind a TLS-terminating proxy.
- SMTP vs HTTP email: why Railway blocking SMTP forces the Brevo HTTP-API design.

🔑 Pro insights

> Two kinds of "env var." Backend secrets (DATABASE_URL, BREVO_API_KEY, INBOUND_WEBHOOK_TOKEN, ADMIN_PASSWORD) are injected at runtime by Railway, never reach the client. Frontend VITE_API_BASE is baked into the static bundle at build time and is fully public (fine - it's only a URL). Never put a secret in a VITE_ var.
>
> The CORS step is load-bearing and silent when wrong. Wrong FRONTEND_URL → every ticket submit fails in the browser while curl/Postman still work. That asymmetry is the tell. No-framework, hand-authored MPA. Nav, footer, and download bar are physically duplicated across all four HTML files (the STYLEGUIDE says "copy exactly"). Editing shared chrome means editing four files.
>
> db push in prestart ships schema changes with no migration history and no rollback. Fast for a solo project, but no safety net if a prod change must be reversible. Committed binaries violate "never commit build artifacts" on purpose - the backend is the download CDN. Cost: every product rebuild needs the fresh .apk/.exe copied into backend/downloads/ and committed, or the site distributes a stale build. (assetlinks.json also still has a placeholder SHA-256 - Android App Links won't verify until filled in.)
>
> The keepalive is "good-enough for free tier," not an SLA. Scheduled runs are best-effort, can be delayed, and only run from the default branch - so the occasional cold start still slips through. The doc suggests UptimeRobot as a sturdier backstop.

Self-check

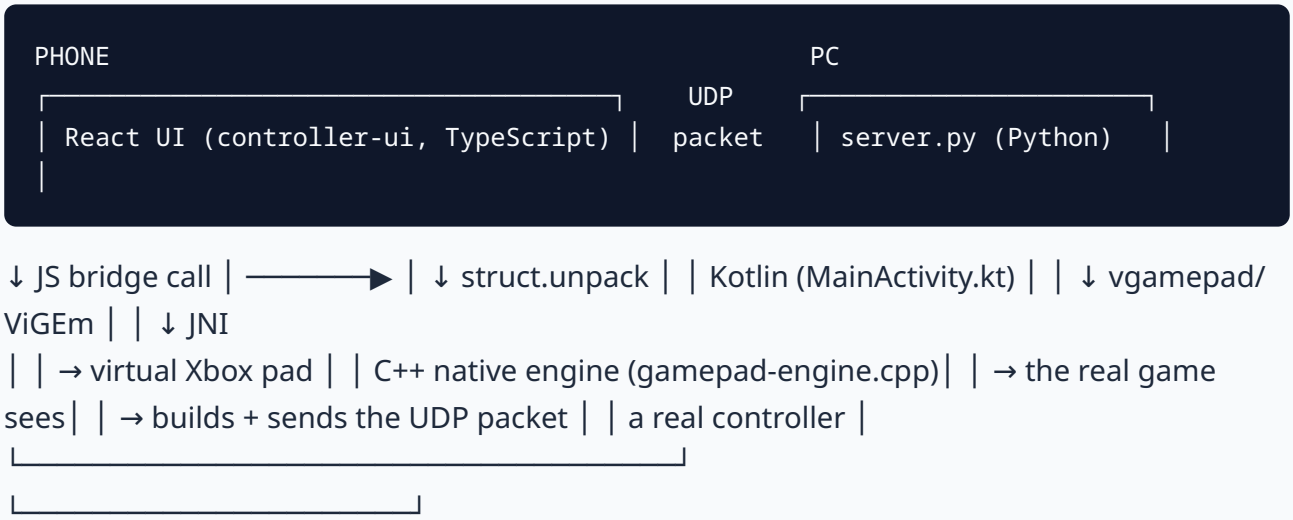
- Why does a ticket submit fail in the browser but succeed in curl when FRONTEND_URL is wrong?
- Why does changing the backend URL require a frontend rebuild?
- Which env vars are public and which are secret, and why?
- What's the keepalive Action's purpose and its honest limitations?

PHASE 7 — THE PRODUCT: PHONE → PC INPUT PATH

Goal: Understand how a touch on the phone screen becomes a button press in a PC game, end to end, across three languages.

The mental model

The product is three programs talking to each other, all under F:/hlooo/apps/:



- apps/controller-ui/ - the on-screen buttons/sticks/D-pad, written in React + TypeScript. This is what you see. It doesn't touch the network directly - it calls into the native layer through a JavaScript bridge object (window.AndroidBridge, window.sendGamepadPacket, etc.), the same way a webpage calls a browser API.
- apps/android-client/ - the native Android shell. MainActivity.kt hosts the WebView (running the React UI above), exposes the JS bridge methods, and manages sensors, USB, and the app lifecycle. app/src/main/cpp/gamepad-engine.cpp is the actual C++ networking engine - it owns the UDP socket, builds the 20-byte packet, and runs on its own background thread so a busy UI never delays a button press.
- apps/pc-server/ - server.py, a Python program that listens for UDP packets, decodes them, and feeds a virtual Xbox controller (via a library called vgamepad, backed by a Windows driver called ViGEm). To Windows and every game, this virtual pad is indistinguishable from a real USB controller.

The wire format (the 20-byte packet) — read & trace

This is the shared contract between all three programs. If one side changes it without the others, input breaks - sometimes silently.

Bytes	Field	Meaning
0–7	timestamp (uint64)	when the phone sent it - used to measure round-trip latency
8–9	buttons (uint16)	one bit per button, packed
10	left trigger (uint8)	
0–255, analog		
11	right trigger (uint8)	0–255, analog
12–13		
left stick X, Y (uint8 each)	128 = center	
14–15		
right stick X, Y (uint8 each)	128 = center	
16–19	auth token (uint32)	
proves the packet came from a paired phone		

Find this format in three places and compare them: App.tsx (DataView.setUint8/setUint16 calls building the packet), gamepad-engine.cpp (the GamepadPayload C struct, #pragma pack'd to exactly 20 bytes), and server.py (PAYLOAD_FORMAT = '<Q H B B B B B B I' - Python's struct module format string). All three must agree byte-for-byte.

Hands-on exercises

1. Trace one button press. Find Btn in Widgets.tsx (onPointerDown), follow it to dn(id) in App.tsx, then to sendGamepadTelemetry, then to the packet's bitmask byte, then to injectNativePayload in gamepad-engine.cpp, then to the TX thread's send()/sendto() call.
2. Regenerate the wire-format comparison. Grep each of the three files for the format string/struct and copy them side by side - confirm they still match after any future edit.
3. Run the self-tests. cd apps/pc-server && python grx_crypto.py and python grx_session.py - both print ALL ... TESTS PASSED when the crypto is intact.

🔑 Pro insights

> The C++ engine runs on its own thread, not the UI thread. A dedicated TX (transmit) thread means a laggy WebView render never delays an outgoing packet - this is why input stays responsive even while the UI is busy animating something. Two transports exist: UDP (Wi-Fi/USB-tether) and AOA (Android Open Accessory, direct USB). AOA bypasses the whole IP stack for the lowest possible latency but needs a driver-level USB handshake - it's the "advanced" path; UDP is the default that always works.

PHASE 8 — THE PC SERVER & GRX ENCRYPTION

Goal: Explain how the Python server drives a virtual controller, and how the GRX encrypted-input layer works well enough to reason about a handshake failure.

server.py — the shape of it

- Opens a UDP socket, listens for packets.
- For each source IP, PadManager.acquire() creates (once) a vg.VX360Gamepad() - the virtual controller - and keeps a session per phone (so multiple phones can each drive their own pad).
- Unpacks the 20-byte packet, applies button/stick/trigger state to the virtual pad, sends it an ACK (so the phone can measure round-trip time), and - if the game requested force-feedback - sends a rumble packet back.
- The PyInstaller spec file (GamepadServer.spec) tells PyInstaller which Python modules to bundle into the final .exe - if you add a new import (like the cryptography package for GRX) and forget to add it to the spec's hiddenimports, the built exe silently lacks it and crashes/misbehaves at runtime with no compile-time warning. This exact mistake happened during GRX development - the shipped exe was rebuilt from before GRX existed, so it couldn't answer the new encrypted handshake at all. Lesson: a "successful build" of the client doesn't mean the server you already shipped can talk to it.

GRX — the encrypted input layer, explained simply

Before GRX, the 20-byte packet traveled in plaintext with only a simple numeric "auth token" as protection - anyone sniffing the same Wi-Fi network could read and forge your input. GRX fixes this with real cryptography:

1. Handshake (once per connection): phone and PC each generate a temporary key pair (X25519, a well-studied key-exchange algorithm), exchange public keys, and derive a shared secret without ever transmitting the secret itself. This shared secret plus the phone's pairing key (from the QR code) is fed through HKDF (a standard way to turn one secret into several separate, purpose-specific keys) to get one key for phone→PC traffic and a different one for PC→phone traffic.

2. Per-packet encryption: each 20-byte input frame is sealed with AES-GCM, an authenticated encryption mode - it both hides the contents and proves nobody tampered with it, growing the packet from 20 to 41 bytes (the extra bytes are the encryption "tag" plus a counter).
3. Replay protection: each sealed packet carries an ever-increasing counter; the receiver tracks which counters it's already seen and rejects duplicates or replays.
4. Graceful fallback: if the PC server doesn't understand the encrypted handshake (e.g. it's running an old build), the phone's handshake attempt is just silently ignored by the old server, and the phone falls back to sending plaintext - so pairing still works, just without encryption, rather than breaking entirely.

Hands-on exercises

1. Read the wire diagram. apps/docs/GRX_PROTOCOL.md documents the exact byte layout of every handshake message.
2. Run the crypto self-tests (grx_crypto.py, grx_session.py) and read what each assertion proves (round-trip works, replay is rejected, a wrong key is rejected, tampering is detected).
3. Find the version mismatch. Compare the file dates of dist/GamepadServer.exe against grx_crypto.py/grx_session.py/server.py - if the exe is older, it was built before GRX existed and won't speak it. This is a real, recurring release hazard for this project: PyInstaller must be re-run after every server-side source change.

🔑 Pro insight

> Encryption without a keeper for the past is dangerous - but so is not shipping the fallback. GRX is deliberately optional: GRX_REQUIRED=False on the server means old clients keep working. Once you're confident every deployed client speaks GRX, flipping that to True retires the plaintext path for good.

PHASE 9 — GYRO STEERING: TWO MODES, ONE SENSOR

Goal: Explain why raw gyroscope math is harder than it looks, and how this project's two gyro modes (Racing vs. 3D) work.

The problem with the "obvious" approach

A naive way to turn phone tilt into steering is to ask Android for the phone's Euler angles (roll/pitch/yaw - think "how far turned, tipped, spun"). This sounds right but has a landmine: at certain orientations (specifically when the phone is held roughly vertical, facing the player - a completely normal way to hold it for steering), the roll and yaw axes mathematically collapse into each other. This is gimbal lock, and it's exactly the bug this project hit: steering would "lock up" partway through a tilt and the effective sensitivity would change depending on how upright the phone was held.

The fix: read gravity directly, not Euler angles

Instead of asking "what's my roll angle," the fixed code asks a more stable question: "which way is down, described in the phone's own left/right and up/down axes right now?" That's read straight from the sensor-fusion rotation matrix (a 3×3 grid of numbers Android's SensorManager produces from combining the gyroscope and accelerometer) - specifically the row that represents "world up," projected onto the phone's screen plane. The angle of that projection (atan2, a trigonometry function for "angle of this 2D vector") is a stable, gimbal-lock-free measurement of left/right tilt, because both of its inputs stay large (not near-zero) throughout a normal steering-hold range.

Two modes, one underlying measurement

The sensor produces two independent, gravity-stable numbers: roll (left/right tilt) and pitch (forward/back tilt). The app exposes two ways to use them: - Racing mode - only roll matters. It drives the left stick's X axis (like a wheel), and the on-screen tilt bar indicator is shown. - 3D mode - both roll and pitch matter, driving the right stick's X and Y (like a look/aim stick in a flight or 3rd-person game). The tilt bar is hidden (it's meaningless with two axes); a small "GYRO 3D" chip shows instead. A Calibrate button lets the player set "center" at whatever

angle is comfortable, since - unlike racing, which has a natural upright reference - there's no single "correct" hold angle for looking around. A deliberate simplification: true 3D orientation has a third axis, yaw (turning left/right around a vertical axis, like spinning in your chair). This project intentionally does not use yaw for steering, because measuring it accurately requires a magnetometer (compass) reading, which drifts and isn't reliable indoors. Two stable axes beat three drifty ones.

Hands-on exercises

1. Read onSensorChanged in MainActivity.kt. Find the remapCoordinateSystem call (normalizes for phone orientation), then the atan2 calls that compute roll and pitch.
2. Read the mode-routing in App.tsx. Find the if (gyroMode === "racing") branch and see how the same underlying tilt values feed completely different stick axes depending on mode.
3. Trigger the diagnostic log. adb logcat -s GYRO while tilting the phone - watch the printed roll/pitch values change live, and confirm the sign matches your physical tilt direction (this project's team used exactly this technique to catch and fix an inverted-direction bug).

🔑 Pro insight

> When a sensor reading feels "unstable" or "wrong direction," suspect the math before the hardware. This project's steering bug wasn't a bad phone or bad sensor - it was a formula (Euler angles) that's mathematically unstable at a specific, very-common orientation. Switching to a differently-shaped formula (gravity projection) that avoids the unstable region fixed both the direction and the smoothness in one change.

PHASE 10 — SHIPPING A RELEASE (THE REAL CHECKLIST)

Goal: Follow the full path from "I changed some code" to "a user's phone offers them an update" without breaking anyone's install.

> The authoritative, always-current version of this checklist lives in F:/hlooo/RELEASE.md - read this phase as the narrated, explained version of that file.

The three things that must move together

Every release touches: (1) the actual build artifact (APK or EXE), (2) the version numbers baked into that artifact, and (3) the record on the backend that tells the in-app updater "this is the current version, here's its checksum." Miss one and something subtle breaks - usually not immediately, but the next time someone tries to update.

Android release steps (see RELEASE.md "Android" section for exact commands)

1. Bump versionCode (integer, always increases) and versionName (human string) in app/build.gradle.kts.
2. If the React UI (controller-ui) changed, rebuild it (npm run build) and copy dist/ into the Android app's assets folder - the APK bundles a snapshot of the UI; forgetting this step ships stale UI inside a "new" build.
3. Build the signed release APK (gradlew assembleRelease) - must use the same signing key as every prior release, or the update install is rejected on-device.
4. Copy it into website/backend/downloads/ with a version-unique filename (never overwrite - old versions stay downloadable).
5. Activate it in the admin portal's Releases panel (needs a RELEASE_KEY secret) - this is the step that actually flips what /api/version reports, which is what the in-app updater checks. Nothing above this line is visible to users until this happens.

6. Verify: hit /api/version, confirm the new numbers; on a device with the old version, confirm "Update available" appears and installs cleanly.

PC server release steps

1. Bump the version in three synced places: server.py's APP_VERSION, and the installer script's AppVersion + VersionInfoVersion (installer/GamepadServer.iss).
2. Rebuild the raw exe (pyinstaller GamepadServer.spec), then rebuild the installer (installer/build-installer.ps1, which wraps that exe with the ViGEmBus driver + firewall setup via Inno Setup).
3. Compute the installer's SHA-256 checksum and update the trust-callout on the website (so a technical user can verify the download hasn't been tampered with).
4. Copy into website/backend/downloads/ with a unique name, then activate in the admin portal (same as Android).

Why a third-party app store (Uptodown) matters here

Direct APK downloads trip security warnings on Android because the OS doesn't recognize the publisher the way it recognizes an established store. Getting listed on a verified store like Uptodown - which reviews and certifies apps - gives users a trusted alternative, especially useful while a small/new app is still building a reputation. This project links Uptodown from the website (hero button, a download-page badge, and an interstitial that reminds users the direct APK is the early-access build while Uptodown carries the fully-tested release) - and had to fix a real rejection along the way (the initial submission's icon was too small, at 128×128 instead of the required 256×256+).

Hands-on exercises

1. Read RELEASE.md top to bottom and, for each of the "three synced places" for the PC server, find the exact line in the exact file.
2. Simulate a version mismatch on purpose (in a local/test copy): bump server.py's APP_VERSION but not the .iss file, rebuild only the exe, and observe what the admin Releases panel shows (a ? where the version should be) - then fix it.
3. Walk the "why can't I just copy the file" question. Explain out loud why dropping a new .apk into downloads/ isn't enough by itself - trace exactly which backend record actually controls what the updater offers.

🔑 Pro insight

> "It builds" and "users can get it" are two different finish lines, separated by an explicit activation step on purpose - so a build that compiles but hasn't been tested on a real device never accidentally reaches every user the moment gradlew finishes. Treat "activate" as a deliberate, reversible decision, not a formality.

DEEP-DIVE REFERENCE SECTIONS

Internalize these until you can reproduce them from memory.

A. Every API endpoint — the reference table (verified 2026-06-21)

Method	Path	Auth	Purpose	Line
GET	/	none	Health check (keepalive target)	1115
GET	/api/version	open (* CORS)	Version manifest + download URLs	1056
POST	/api/support/ticket	none (public)	Create a support ticket	506
GET	/api/csat?ticketId=&rating=	none	Capture CSAT rating → HTML page	1063
POST	/api/csat/:ticketId/feedback	none	Free-text CSAT follow-up	1081
GET	/api/admin/auth-state	none	Setup-vs-login mode (reveals admin count)	311
POST	/api/admin/setup	none, gated empty-table	Create first owner	328
POST	/api/admin/login	none	Login, mint session	352
POST	/api/admin/logout	session	Delete session, clear cookie	373
GET	/api/admin/me	requireAdmin	Current admin identity	381
POST	/api/admin/me/password	requireAdmin (real id only)	Change own password	387
POST	/api/admin/upload	requireAdmin	Multipart upload → downloads/	302
GET	/api/admin/team	requireAdmin (masked for agents)	List team	404
POST	/api/admin/team	requireAdmin + requireOwner	Add admin (P2002→409)	421
PUT	/api/admin/team/:id	requireAdmin + requireOwner	Update role/active/reset pw	441

Method	Path	Auth	Purpose	Line
DELETE	/api/admin/team/:id	requireAdmin + requireOwner	Delete admin (last-owner guard)	473
GET	/api/admin/audit	requireAdmin + requireOwner	Activity log	493
GET	/api/admin/tickets	requireAdmin	List tickets (+messages, assignee)	554
PUT	/api/admin/tickets/:id	requireAdmin	Update status/priority/tags (+status email, CSAT if resolved)	577
POST	/api/admin/tickets/:id/reply	requireAdmin	Reply (awaited email + thread + CSAT if resolved)	628
PUT	/api/admin/tickets/:id/assign	requireAdmin	Assign / take / unassign	690
POST	/api/admin/tickets/:id/read	requireAdmin	Clear hasNewReply	719
POST	/api/admin/tickets/:id/note	requireAdmin	Internal note	839
POST	/api/admin/tickets/:id/presence	requireAdmin	Presence heartbeat (write)	858
GET	/api/admin/tickets/:id/presence	requireAdmin	Presence (read)	863
DELETE	/api/admin/tickets/:id	requireAdmin + requireOwner	Delete ticket (cascades messages)	927
GET	/api/admin/canned	requireAdmin	List canned responses	883
POST	/api/admin/canned	requireAdmin	Add canned response	889
DELETE	/api/admin/canned/:id	requireAdmin	Delete canned response	899

Method	Path	Auth	Purpose	Line
GET	/api/admin/settings	requireAdmin	Read settings	910
PUT	/api/admin/settings	requireAdmin + requireOwner	Update settings	914
GET	/api/admin/contacts	requireAdmin	Broadcast recipients	973
POST	/api/admin/broadcast	requireAdmin + requireOwner	Mass email (queued, conc 6, cap 500)	1002
GET	/api/admin/email-status	requireAdmin	Verify Brevo key (/v3/account)	942
POST	/api/admin/email-test	requireAdmin	Send test email	961
GET	/api/admin/stream	requireAdmin (Bearer reaches it)	SSE live event stream	868
POST	/api/email/inbound	URL token	Brevo inbound webhook	738
GET	/admin	session redirect	Serve admin SPA	1100
GET	/admin/login	session redirect	Serve login HTML	1107
*	(anything else)	-	404 JSON catch-all	1120

> Drill: cover the right two columns and reconstruct the auth + line for each route from memory. Re-run the Phase-0 grep after any edit - these line numbers drift.

B. The auth model — recite this

Passwords: scrypt-hashed with a 16-byte per-password salt (scrypt:salt:hash, auth.js), verified with timingSafeEqual. Login mints a 256-bit opaque token, stores an AdminSession (30-day TTL), sets an HttpOnly/SameSite=Lax/Secure cookie gp_admin. Every request: requireAdmin → session lookup joined to Admin, checks not-expired AND admin.active, sets req.admin. requireOwner layers a role check. A Bearer ADMIN_PASSWORD header is a synthetic-owner back door (id:null, plain ===, reaches even the SSE stream, no audit trail). Weaknesses: Bearer

back door, no rate limiting/lockout, no CSRF tokens (only SameSite=Lax), conditional Secure flag, unaudited failed logins.

C. The webhook — recite the round-trip

Outbound stamps Reply-To: ticket+@ + threading headers (+ CSAT links if resolving). User replies → MX routes to Brevo (SMTP) → Brevo parses → HTTP POST to /api/email/inbound?token=.... Token check (401/503) → ack 200 first → background match in 3 tiers (UUID in recipient → [#ref] in subject → newest ticket by sender email) → no match forwards to admin → primary write (message + hasNewReply + reopen-if-resolved) → best-effort lastEmailMessageId write → SSE broadcast. No idempotency; ack-first means failed saves are lost; split write defends against missing columns.

D. The data model relations — recite

- Ticket 1→* TicketMessage (Cascade - delete ticket deletes its messages).
- Admin 1→* AdminSession (Cascade - delete admin deletes sessions).
- Admin 1→* Ticket via named "AssignedTickets" (SetNull - delete admin unassigns tickets).
- Admin 1→* AuditLog (SetNull + denormalized adminName - delete admin preserves history).
- Setting (key/value, key is PK). CannedResponse (standalone). AdminSession token is PK. All IDs UUID except Setting (key) and AdminSession (token).

MASTERY CHECKLIST — BEAST-LEVEL COMMAND

You have BEAST-level command when every one of these is true, without notes:

End-to-end build capability - [ ] Add a column to a Prisma model, db push it, write it in an authenticated API route, return it in the {success:true,...} contract, and surface it in admin.html with optimistic update + rollback. - [ ] Add a new authenticated API endpoint (chaining requireAdmin/requireOwner), persist via Prisma, handle P2002/P2025, and explain its full lifecycle from CORS to response. - [ ] Wire a new [data-dl] or contact-form interaction and explain the cross-origin contract it depends on.

Request lifecycle & routing - [ ] Trace POST /api/admin/tickets/:id/reply line by line: middleware order, validation, nested write, awaited email, audit, SSE, response. - [ ] Explain why the 404 catch-all must be last and why body parsing precedes routes.

Auth - [ ] Recite the full login→request→logout flow and every DB write. - [ ] Name three real weaknesses (Bearer back door incl. SSE reach, no rate limiting, no CSRF) and a concrete fix for each. - [ ] Explain why trust proxy is mandatory and what breaks without it.

Webhook / email / CSAT - [ ] Explain the SMTP vs HTTP hops, the 3-tier matching, why ack-first, and why the write is split. - [ ] Simulate a Brevo payload with curl and hit all three tiers. - [ ] Explain which emails are awaited vs fire-and-forget and why. - [ ] Trace the full CSAT round-trip (email link → csatRating → feedback form → csatFeedback).

Data model - [ ] Draw the ER diagram of all models from memory with every onDelete rule. - [ ] Explain what deleting an admin does to their sessions, tickets, and audit rows.

Admin portal - [ ] Explain the api() wrapper, optimistic updates, SSE+poll, the esc() trust boundary, and the broadcast worker pool (concurrency 6, cap 500).

Infra / deploy & security - [ ] Diagnose a "works in curl, fails in browser" CORS bug in under a minute. - [ ] Explain build-time vs runtime env vars and which must never hold secrets. - [ ] Explain the keepalive Action's purpose and limitations. - [ ] Explain the public-attachment exposure (/downloads is unauthenticated) and how you'd fix it; identify that uploads/ is dead.

The product (Phases 7–10) - [ ] Trace one button press from a touch on the React UI to the packet leaving the phone's UDP socket, naming every file and language crossed. - [ ] Recite the 20-byte packet layout from memory and explain why all three implementations (TS/C++/Python) must agree byte-for-byte. - [ ] Explain GRX's handshake (X25519 → HKDF → per-direction AES-GCM keys) well enough to diagnose "the encrypted handshake never completes" as either a stale server build or a real crypto mismatch. - [ ] Explain gimbal lock in one sentence and why the gravity-projection fix avoids it. - [ ] Explain the difference between Racing mode (roll → left stick) and 3D mode (roll+pitch → right stick, plus Calibrate) and why yaw is deliberately excluded. - [ ] Walk a full release from source change to a real device offering "Update available," naming every version-number location that must move together.

The non-obvious (competent → BEAST) - [ ] Know which decisions are forced by the host (Railway blocks SMTP → Brevo HTTP API; trust proxy; db push defensiveness) vs chosen for design. - [ ] Understand the single-instance assumptions (in-memory presence + settings cache) and what breaks under horizontal scaling. - [ ] Articulate the db push-drift hazard and point to every place the code defends against it.

CADENCE, ORDER OF ATTACK, AND HONEST DIFFICULTY

Order: Phase 0 → 1 → 2 → 3 → 4 → 5 → 6. Dependency-ordered: data is the vocabulary, the API is the spine, auth guards it, the portal consumes it, the webhook/frontend/infra are the edges. Don't skip Phase 1 - every later phase assumes it.

Phase	Subject	Effort	Calendar
0	Foundations & orientation	~3–4 h	Days 1–2
1	Data layer (Prisma)	~6–8 h	Week 1
2	API server core	~8–10 h	Week 2
3	Auth & authz	~6–8 h	Week 3
4	Admin portal SPA	~8–10 h	Week 4
5	Inbound webhook + CSAT	~8–12 h	Week 5 (give it room)
6	Frontend + infra/deploy	~5–7 h	Week 6
7	The product: phone→PC input path	~4–6 h	Week 7
8	PC server + GRX encryption	~5–7 h	Week 7–8
9	Gyro steering (the math)	~3–4 h	Week 8
10	Shipping a release	~3–4 h	Week 8
-	Mastery drills + reference recall	ongoing	Week 9+

~8–9 weeks to BEAST level across the whole project at a sustainable ~6–8 h/week; the original website-only plan (Phases 0–6) is still ~6–7 weeks on its own if that's all you need right now.

Hardest (and why): 1. The inbound webhook (Phase 5) - two protocol hops, three matching tiers, ack-first reliability, split-write defensiveness. Several behaviors only make sense once you know the past bugs that shaped them. 2. Auth's subtle edges (Phase 3) - the mechanics are mechanical; the id:null Bearer back door and the security reasoning take real thought. 3. The admin portal's optimistic updates + SSE-vs-poll + no-diffing innerHTML guards (Phase 4) - easy to read, hard to fully reason about.

Easier than it looks: the API "38 routes" is the same handful of patterns (validate → Prisma → audit → SSE → respond) repeated. Truly trace the reply flow and every other route is readable. The marketing site is almost entirely static - the only real logic is one fetch.

You built this. By the end you'll understand it better than anyone alive - exactly where a solo founder needs to be. Go.